Shree Rahul Education Society's (Regd.)

SHREE L. R. TIWARI COLLEGE OF ENGINEERING

An Autonomous Institute Affiliated to University of Mumbai,

Approved by AICTE & DTE, Maharashtra State | NAAC Accredited, NBA Accredited Program | ISO 9001:2015 Certified | DTE Code No: 3423 | Recognized under Section 2(f) of the UGC Act 1956 | Minority Status (Hindi Linguistic)

DEPARTMENT OF COMPUTER ENGINEERING

CML610 Cloud Computing Lab

Sixth Semester, 2025-2026 (Even Semester)

Name of Student : Ayan Parkar

Roll No. : 48

Division : A

Batch : A2

Day / Session :

Venue :

Experiment No. : 5

Title of Experiment : To study and Implement Platform as a Service using AWS Elastic Beanstalk.

Date of Conduction :

Date of Submission :

Particulars	Max. Marks	Marks Obtained
Preparedness and Efforts(PE)	**3**	
Knowledge of tools(KT)	**3**	
Debugging and results(DR)	**3**	
Documentation(DN)	**3**	
Punctuality & Lab Ethics(PL)	**3**	
Total	**15**	

Grades – Meet Expectations (3 Marks), Moderate Expectations (2 Marks), Below Expectations (1 Mark)

Checked and Verified by

Name of Faculty : Pravin Jangid

Signature :

Date :

Experiment No 5

Aim: To study and Implement Platform as a Service using AWS Elastic Beanstalk.

Theory:

1. Prepare a detailed study of Platform as a Service

   a. What is PaaS?
   PaaS bundles core cloud infrastructure—such as servers, networking, and storage—with platform-specific tools like operating systems, runtime environments, databases, and middleware. The entire environment is delivered over the internet by a third-party provider, who assumes responsibility for hardware maintenance, system updates, and load balancing. This abstraction accelerates software delivery by removing the operational friction of complex backend setups

   b. How to use PaaS?
   Developers interact with a PaaS through a graphical user interface (GUI), command-line interface, or API to write and test code using built-in SDKs. To deploy applications, teams simply push their code to the platform, which automatically handles the hosting, runtime execution, and operating system management. Users can also configure automated continuous integration and delivery (CI/CD) pipelines, set up auto-scaling rules for traffic spikes, and seamlessly connect microservices.

   c. How to provide PaaS?
   Providing a PaaS requires architecting a multi-layered cloud environment that abstracts backend complexity from the end user. Providers first establish a robust infrastructure layer utilizing virtual machines or container orchestration systems like Kubernetes, and then integrate automated provisioning, middleware, and database services. Finally, providers expose these capabilities through developer-friendly portals and APIs, ensuring secure resource allocation, multitenancy, and automated lifecycle management.

2. Advantages and Limitation of PaaS

Operational Aspect	Advantages	Limitations
Development & Deployment	Accelerates time-to-market with pre-built frameworks, automated CI/CD pipelines, and isolated testing environments speridian+2.	Requires staff with specialized knowledge to master the provider's specific tools and environment iiot-world.
Cost Management	Eliminates upfront hardware investments through a pay-as-you-go model and reduces the need for dedicated IT maintenance staff geeksforgeeks+2.	Deep integration with a specific provider's tools can lead to severe vendor lock-in, making future migrations difficult blog.back4app+1.
Scalability & Performance	Provides automatic scaling and load balancing to dynamically handle traffic spikes without manual intervention milvus+1.	Providers often enforce resource caps on CPU and memory, which can bottleneck applications requiring heavy processing geeksforgeeks+1.
Infrastructure & Maintenance	The provider completely manages the underlying infrastructure, including automatic security patches and system updates blog.back4app+1.	Offers limited flexibility, as built-in tools and shared resources may not support highly customized or unique business requirements geeksforgeeks+1.

3. Study security issues in PaaS

While Platform as a Service (PaaS) accelerates development and abstracts infrastructure management, it introduces unique security challenges that organizations must address. Because PaaS relies on a shared responsibility model, cloud providers secure the underlying hardware and infrastructure, but users remain responsible for securing their applications, configurations, and access controls.

For resource efficiency, PaaS providers often run applications from multiple different customers on the same operating system instance or server, relying on containers or language-specific sandboxes (like the Java Virtual Machine) to keep them isolated. A critical security risk is "container breakout," where an attacker exploits a vulnerability to escape their isolated environment and gain root privileges, potentially accessing other tenants' sensitive data or compromising the entire underlying host.

Because PaaS platforms rely heavily on Application Programming Interfaces (APIs) for provisioning, management, and integration, these interfaces are a primary attack vector. Poorly secured APIs may suffer from inadequate authentication, insufficient input validation, or a lack of rate-limiting. Attackers can exploit these flaws through techniques like SQL injection or cross-site scripting (XSS), allowing them to manipulate databases or bypass security protocols.

One of the most common causes of data exposure in PaaS environments is user misconfiguration. Administrators may inadvertently grant excessive permissions, leave storage buckets unsecured, or fail to implement outbound traffic restrictions. Without proper identity and access management (IAM) and strict zero-trust principles, attackers or internal actors can exploit these misconfigured access controls to exfiltrate sensitive data without detection.

PaaS providers manage the platform, but developers are entirely responsible for the code they deploy on it. If development teams lack security expertise or fail to use secure coding practices, they introduce application-level vulnerabilities into the ecosystem. Furthermore, the complexity of modern cloud architecture and the integration of third-party open-source dependencies can create supply chain vulnerabilities, making it difficult to detect legacy code flaws before deployment.

Because PaaS abstracts the underlying infrastructure away from the user, it inherently reduces visibility into network traffic and system-level security events. This "visibility void" means organizations often lack the traditional security telemetry needed to quickly detect intrusions or audit access logs, forcing them to adopt specialized cloud-native monitoring tools to track threats in real-time.

4. Technologies used to provide PaaS

Providing a Platform as a Service (PaaS) requires a multi-layered technology stack that abstracts hardware complexity and delivers a seamless development environment. Providers build this stack by combining underlying cloud infrastructure with containerization, middleware, and automated deployment tools.

At the foundational level, PaaS relies on robust cloud infrastructure, which includes physical servers, storage arrays, and networking equipment. Technologies like virtual machines (VMs) and hypervisors partition the physical hardware, while load balancers and firewalls ensure secure, high-availability traffic routing.

Modern PaaS architectures heavily utilize containers to package applications with their dependencies, ensuring they run consistently across different environments. Providers leverage container engines like Docker alongside orchestration platforms like Kubernetes or Docker Swarm to automate scaling, health checking, and resource allocation. For example, platforms like Cloud Foundry use Kubernetes to manage large-scale deployments, while lightweight options like CapRover rely on Docker Swarm.

To support application development, PaaS providers pre-configure operating systems and language-specific runtime environments for technologies like Python, Node.js, Java, and Ruby. They also integrate middleware technologies such as caching systems, message brokers, and database management engines, which facilitate communication and data storage without requiring manual configuration.

Seamless code deployment is achieved through integrated version control and continuous integration/continuous deployment (CI/CD) pipelines. Technologies like Git enable streamlined deployment models, where developers simply push code to a repository to trigger automated buildpacks that compile the code and launch it into isolated containers.

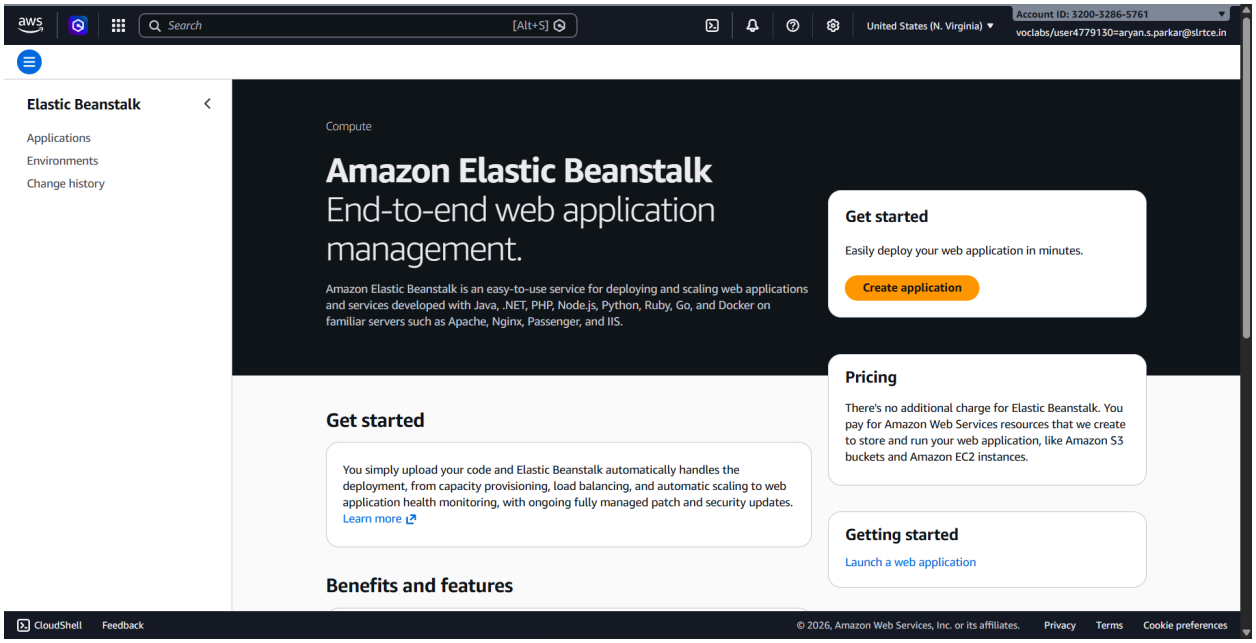
PaaS providers expose their complex underlying technologies through intuitive graphical user interfaces (GUIs), command-line interfaces (CLIs), and APIs. These management layers allow development teams to easily monitor application health, configure auto-scaling rules, and provision secure SSL/TLS certificates using integrated tools like Nginx.

Activity:

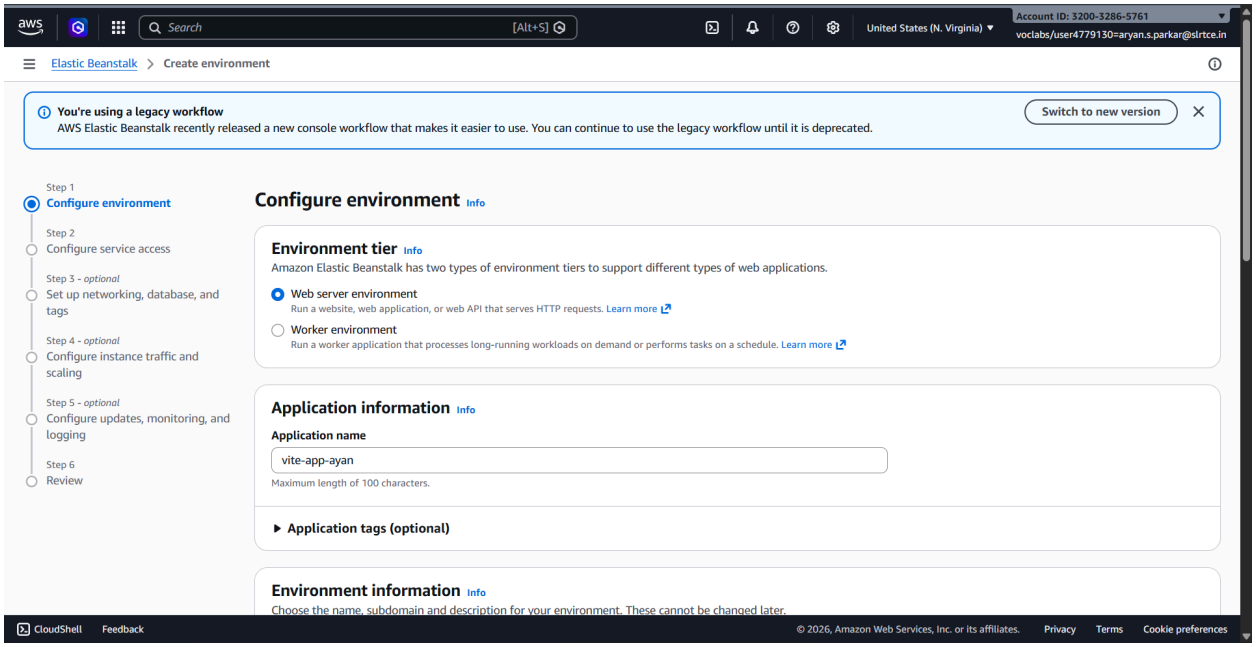
Step 1: Log in to AWS Console

Step 2: Search Elastic Beanstalk, click on it

Step 3: Click on Create Application



Step 4: Configure the application



Step 5: Open your terminal, run the following command

npx create vite

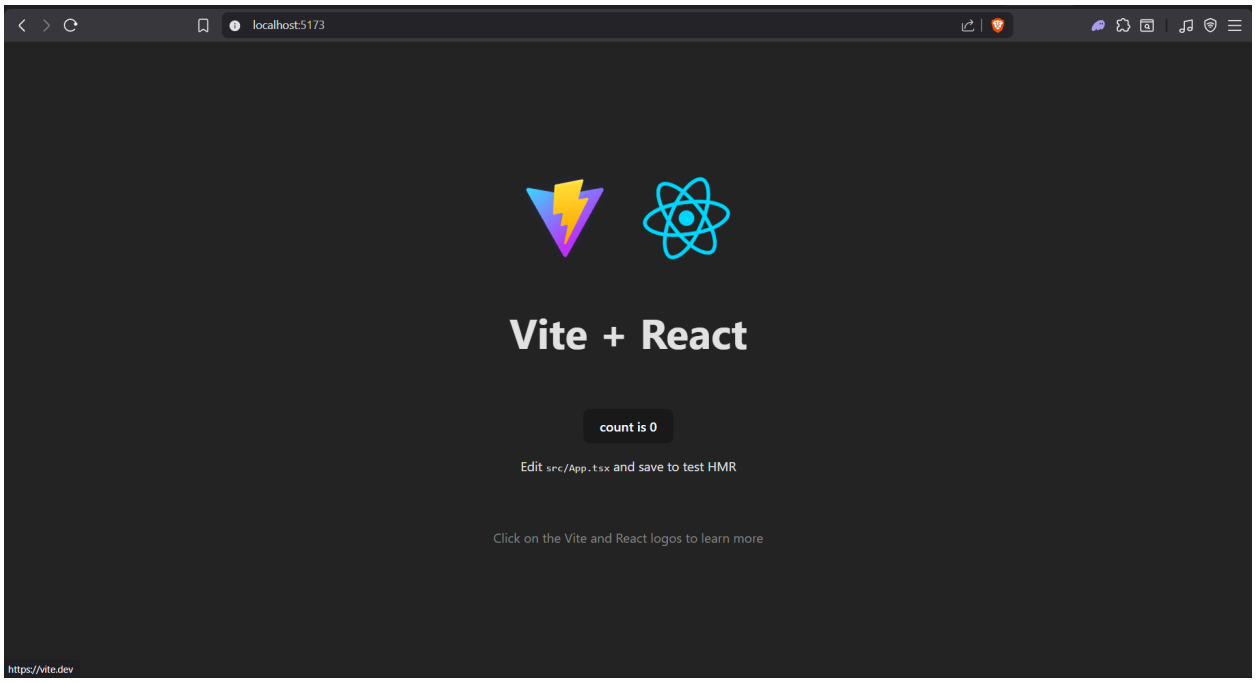
Step 6: Select the React framework and click continue

Step 7: Upon successful initialization, you'll see the following in your terminal

```
VITE v7.3.1  ready in 200 ms

➜ Local:   http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help
```

Step 8: Ctrl + Click on the link

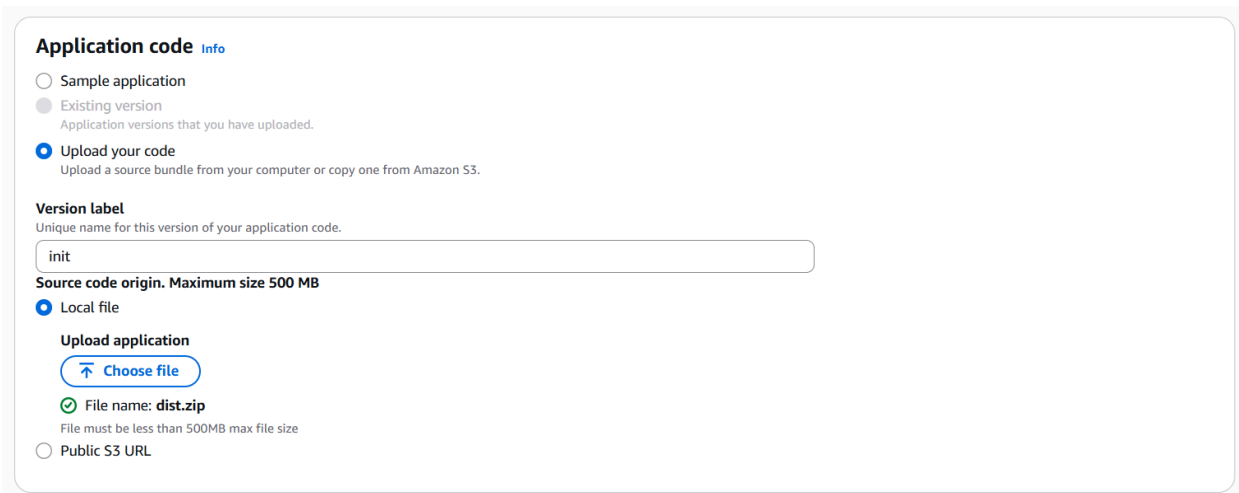


This is the react app running locally, we now have to bundle it and upload it to beanstalk to host it on the cloud

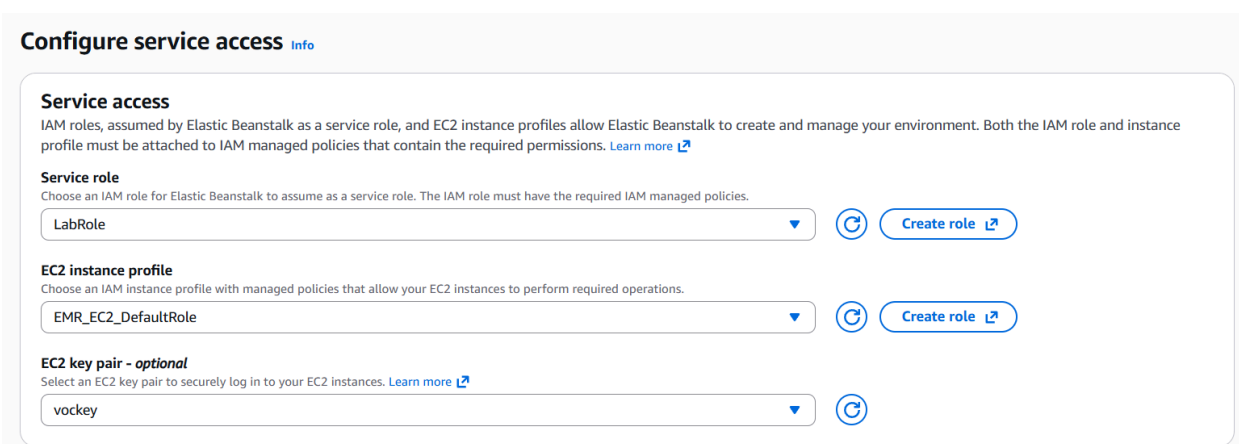
Step 9: Open the terminal again and run the following:

```
ayan@omen16:~$ cd beanstalk-test
ayan@omen16:~/beanstalk-test$ bun run build
$ tsc -b && vite build
vite v7.3.1 building client environment for production ...
✓ 37 modules transformed.
dist/index.html                    0.46 kB │ gzip:  0.30 kB
dist/assets/react-CHdo91hT.svg     4.13 kB │ gzip:  2.05 kB
dist/assets/index-COcDBgFa.css     1.38 kB │ gzip:  0.70 kB
dist/assets/index-DFoPaCK7.js    194.75 kB │ gzip: 61.38 kB
✓ built in 951ms
ayan@omen16:~/beanstalk-test$
```

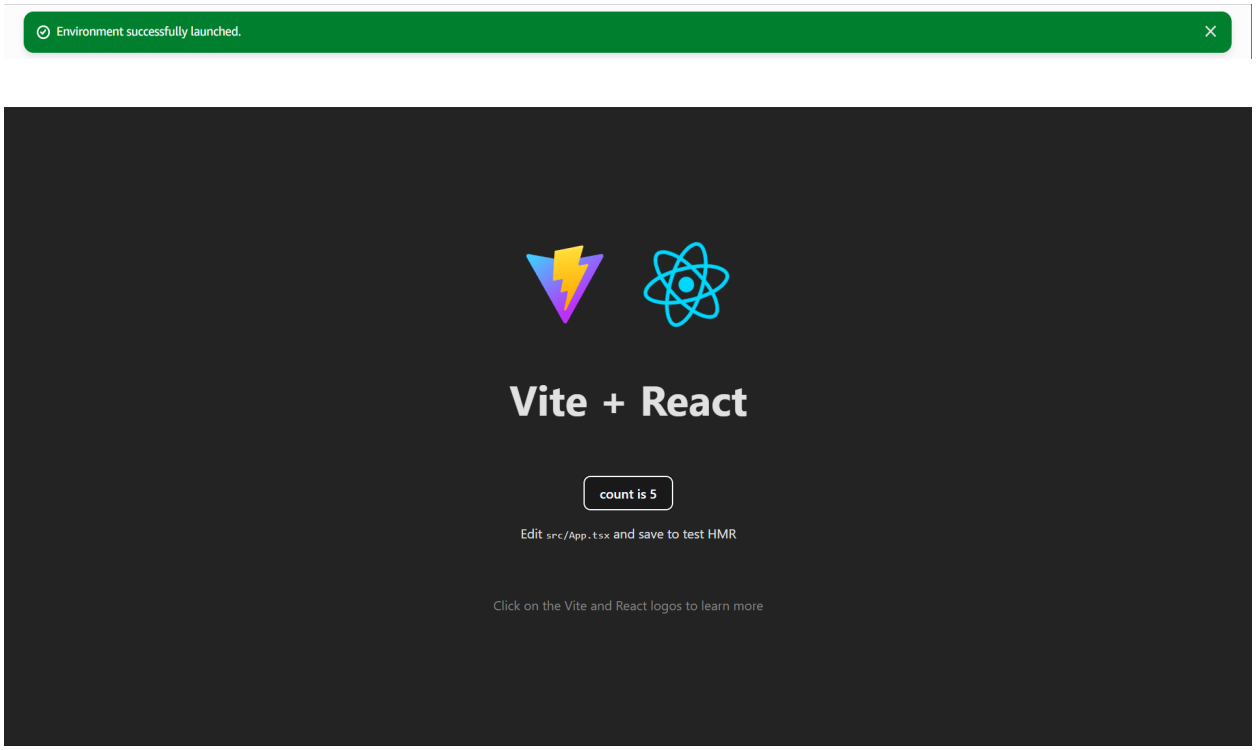
Step 10: The build command generates a dist/ folder, you have to compress the folder into a .zip file and upload it to beanstalk



Step 11: Click on Next and select the following roles:



Step 12: Click on “Skip to review” and then create the application. It will take a few minutes to deploy your app and assign a new domain.



The application has been successfully deployed and can be accessed from any remote machine by entering the mentioned URL of the remote machine’s web browser.

Conclusion:

Platform as a Service (PaaS) provides developers with a complete, managed cloud environment for building, testing, and deploying applications without the burden of managing the underlying infrastructure. By abstracting away hardware, operating systems, and middleware, PaaS enables organizations to focus exclusively on application logic and software development.